ITF MANUAL TESTING COURSE

Computer Store Database

Creating Tables and Writing SQL Queries

Project overview

Item	Details
Student	Valentina Hatos
Role	QA Tester
Application	MySQL database “computerstore”
Module tested	Tables, relationships and SQL queries
Tools	MySQL Workbench
Date	September 2022

Contents

1. Introduction....3

2. Database Structure....3

3. Inserting Data....3

4. Queries....4

4.1 Basic queries....4

4.2 Functions and sorting....5

4.3 Joins and grouping....5

4.4 Changing the data....6

4.5 Combined queries....6

5. Conclusion....7

1. Introduction

This project is an SQL exercise from the ITF Manual Testing course. I created a database for a computer store. The database stores the products of the store and their manufacturers.

I created the tables, inserted data and wrote queries for 25 tasks. For some tasks, I wrote two different solutions to practise more SQL statements.

2. Database Structure

Task 1: Create a database named “computerstore”.

Task 2: Create the tables based on the given schema.

I created the database and two tables: manufacturers and products. I also created the database model in MySQL Workbench.

Table	What it stores
manufacturers	The code and the name of each manufacturer
products	The code, the name, the price and the manufacturer of each product

In both tables, the code column is the primary key. The manufacturer column in the products table is a foreign key. It references the code column in the manufacturers table.

```
create database computerstore;
use computerstore;

create table manufacturers (
code int primary key,
name char(50) not null
);

create table products (
code int primary key,
name varchar(50) not null,
price float not null,
manufacturer int not null,
foreign key (manufacturer) references manufacturers(code)
);
```

3. Inserting Data

Task 3: Insert the given data into the tables.

I inserted 6 manufacturers and 10 products.

Code	Manufacturer
1	Sony
2	Asus
3	Hewlett-Packard
4	Lenovo
5	Fujitsu
6	SanDisk

Code	Product	Price ($)	Manufacturer
1	Hard drive	64.89	5
2	Memory Card	17.99	6
3	Laptop charger	42.29	4
4	USB Flash Drive	10	6
5	Monitor	219.99	1
6	HDMI cable	18.29	2
7	Gaming laptop	2079.59	2
8	Printer	96.32	3
9	Toner cartridge	59	3
10	Computer case	158.92	2

```
insert into manufacturers (code, name) values
(1, 'Sony'), (2, 'Asus'), (3, 'Hewlett-Packard'), ...

insert into products (code, name, price, manufacturer) values
(1, 'Hard drive', 64.89, 5), (2, 'Memory Card', 17.99, 6), ...
```

Tasks 4 and 5: Check that all the records were inserted.

I verified both tables with SELECT * statements. All 6 manufacturers and all 10 products were inserted.

```
select * from manufacturers;
select * from products;
```

4. Queries

4.1 Basic queries

Task 6: Select the names of all the products.

```
select name from products;
```

I selected only one column from the products table.

Task 7: Select the names and the prices of all the products.

```
select name, price from products;
```

I selected two columns.

Task 8: Select the names of the products with a price less than or equal to $200.

```
select name from products
where price <= 200;
```

I used a WHERE condition. The query returns 8 products. Only the Monitor and the Gaming laptop cost more than $200.

Task 9: Select all the products with a price between $60 and $120.

```
select * from products
where price between 60 and 120;

select * from products
where price >= 60 and price <= 120;
```

I wrote two solutions: one with BETWEEN and one with AND. Both return the same two products: Hard drive and Printer.

Task 10: Select the name and the price in cents.

```
select name, price*100 from products;

select name, price*100 as PriceCents from products;
```

I multiplied the price by 100. In the second solution, I used AS to give the column a clear name.

4.2 Functions and sorting

Task 11: Compute the average price of all the products.

```
select avg(price) from products;
```

I used the AVG function. The average price is about $276.73.

Task 11.1 (extra): Compute the average price of the Asus products.

```
select avg(price) from products
where manufacturer = (select code from manufacturers where name = 'Asus');

select avg(price) from products
inner join manufacturers on products.manufacturer = manufacturers.code
where manufacturers.name = 'Asus';
```

I wrote two solutions: one with a subquery and one with INNER JOIN. Both return about $752.27.

Task 12: Compute the average price of all products with manufacturer code 2.

```
select avg(price) from products
where manufacturer = 2;
```

Manufacturer 2 is Asus, so the result is the same as in task 11.1.

Task 13: Compute the number of products with a price larger than or equal to $180.

```
select count(*) from products
where price >= 180;
```

I used the COUNT function. The result is 2.

Task 14: Select the name and price of all products with a price larger than or equal to $180. Sort first by price (descending), then by name (ascending).

```
select name, price from products
where price >= 180 order by price desc, name asc;
```

I sorted by two columns. The Gaming laptop appears first, then the Monitor.

4.3 Joins and grouping

Task 15: Select all the data from the products, including the data of each product's manufacturer.

```
select * from products
left join manufacturers on products.manufacturer = manufacturers.code;
```

I joined the two tables using the foreign key.

Task 16: Select the product name, the price and the manufacturer name of all the products.

```
select products.name, price, manufacturers.name from products
inner join manufacturers on products.manufacturer = manufacturers.code;
```

Both tables have a column called name, so I wrote the table name before each one.

Task 17: Select the average price of each manufacturer's products, showing the manufacturer's code.

```
select avg(price), manufacturer from products
group by manufacturer;
```

I used GROUP BY to compute one average for each manufacturer.

Task 18: Select the average price of each manufacturer's products, showing the manufacturer's name.

```
select avg(price), manufacturers.name from products
inner join manufacturers on products.manufacturer = manufacturers.code
group by manufacturers.name;
```

I combined INNER JOIN with GROUP BY, so the result shows the names instead of the codes.

Task 19: Select the name and price of the cheapest product.

```
select name, price from products
order by price asc limit 1;

select * from products
where price = (select min(price) from products);
```

I wrote two solutions: one with ORDER BY and LIMIT, one with a subquery and MIN. Both return the USB Flash Drive, at $10.

4.4 Changing the data

Task 20: Add a new product: Loudspeakers, $70, manufacturer 2.

```
insert into products
values (11, 'Loudspeakers', 70, 2);
```

I inserted a new row and checked the table with SELECT *.

Task 21: Update the name of product 8 to "Laser Printer".

```
update products
set name = 'Laser Printer'
where code = 8;
```

I used UPDATE with a WHERE condition, so only product 8 was changed.

Task 22: Apply a 10% discount to all products.

```
update products
set price = price - (price*0.1);
```

I used UPDATE without WHERE, so the discount applied to all products.

Task 23: Apply a 15% discount to all products with a price larger than or equal to $120.

```
update products
set price = price - (price*0.15)
where price >= 120;
```

After the 10% discount, three products still cost $120 or more: Monitor, Gaming laptop and Computer case. They received the extra 15% discount.

4.5 Combined queries

Task 24: Select the smallest price of the products with manufacturer 2.

```
select min(price) from products
where manufacturer = 2;
```

I used the MIN function. After the discounts, the smallest price for Asus is about $16.46 (HDMI cable).

Task 25: Select all the products from the manufacturers Sony, Asus and Lenovo.

```
select * from products
where manufacturer in (select code from manufacturers where name in ('Sony',
'Asus', 'Lenovo'));

select * from products
inner join manufacturers on products.manufacturer = manufacturers.code
where manufacturers.name = 'Sony' or manufacturers.name = 'Asus' or
manufacturers.name = 'Lenovo';
```

I wrote two solutions: one with IN and a subquery, one with INNER JOIN and OR. Both return 6 products.

The complete SQL script can be checked in the project repository (computerstore database.sql). The database model is available in computerstore_database_model.mwb.

5. Conclusion

In this exercise, I created a database with two related tables and inserted data into them. Then I wrote queries for 25 tasks. I used filters, functions, sorting, joins, grouping and subqueries. I also changed the data with INSERT and UPDATE statements.

For several tasks, I wrote two solutions. This helped me understand that the same result can be reached in different ways in SQL.